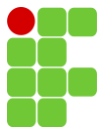


AMBIENTE TINKERCAD – EMULADOR ARDUINO

Sumário

1. Introdução ao Arduino	2
2. Estrutura do Arduino	4
3. Sensores, Atuadores e Módulos	5
4. Diferença entre Emulador e Simulador	6
5. A plataforma TINKERCAD.....	7
6. A Protoboard	11
7. A programação	12
8. Duplicando, Editar e Mover um circuito.....	15
9. Rótulos para comentários	16
10. Os indicadores: LOW e HIGH	17
11. Criação de variáveis	18
12. Monitores de dados	19
13. Acionando um emulador	21
14. Estrutura de um programa ou sketch Arduino.....	22
15. Referências	23



1. Introdução ao Arduino

Arduino é uma plataforma **open-source** de hardware e software para prototipagem eletrônica.

A placa pode tanto receber informações do ambiente ao seu redor através de uma variedade de captadores eletrônicos constituídos por uma infinidade de sensores (iluminação, temperatura, umidade, etc.), quanto enviar comandos para de diversos atuadores (led, buzina, motores, garras controláveis, etc.), bem como estabelecer comunicação com dispositivos de transmissão e/ou recepção, seja via Wifi, Bluetooth, rádio onda ou cabos de rede.



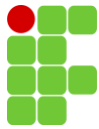
O microcontrolador presente na placa é programado usando a linguagem de programação C e C++.

Os projetos podem ser **standalone** (somente a placa) ou conectados (USB, WIFI, Bluetooth, etc.) com um software rodando em um computador, tablet, smartphone ou outro Arduino.

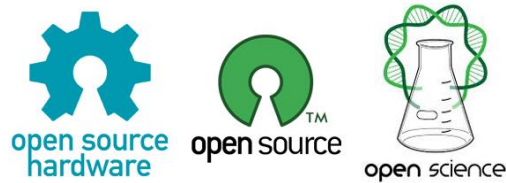
O Arduino é diferente das outras plataformas devido aos seguintes fatores:

- É um ambiente multiplataforma, podendo ser executado em **Windows**, **Macintosh** ou **Linux**.
- Tem por base um ambiente de fácil de programação.
- Pode ser programado utilizando um **cabo de comunicação USB** onde geralmente não é necessária uma fonte de alimentação.
- Possui hardware e software open-source, facilitando a montagem do seu próprio hardware sem precisar de custos excessivos.
- Hardware de baixo custo.
- Grande comunidade ativa de usuários.
- Ambiente educacional, ideal para iniciantes que desejam resultados rápidos.
- Útil para não programadores.

A plataforma atualmente é bastante utilizada em projetos de Computação, Engenharia, Eletrônica, Robótica, bem como para o desenvolvimento de equipamentos meteorológicos, químicos, biológicos e aplicações no ensino de **Física e Ciências**.



Projetos que envolvam o desenvolvimento de códigos e a confecção de **equipamentos laboratoriais de Física** são amplamente criados com o suporte do ambiente Arduino.



Portanto seus recursos abrangem uma ampla diversidade de áreas, como por exemplo: **Computação Física, Design Eletrônico, Arquitetura, Computação Vestível, Telemedicina e Educação.** Sendo sua aplicabilidade generalizada, podemos desenvolver projetos e testá-los com o mínimo ou nenhum conhecimento em programação.



Fonte: www.arduino.cc

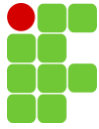
Já o termo **Open Science** (Ciência Aberta) não é um conceito novo em si, embora o acordo sobre este termo e seu uso generalizado seja relativamente recente. Consiste em integrar ou empregar soluções computacionais livres (equipamentos, programas, Objetos de Aprendizagem, etc.) para o ensino e a aprendizagem em Ciências e suas áreas afins.

Muitos outros termos foram usados, e ainda são usados, para se referir à transformação da prática científica, tais como: **Science 2.0** (Ciência 2.0), **e- Science** (e-Ciência), etc. A própria **Arduino.cc** possui uma linha de kits voltadas para Física, o **ARDUINO SCIENCE KIT PHYSICS LAB**.



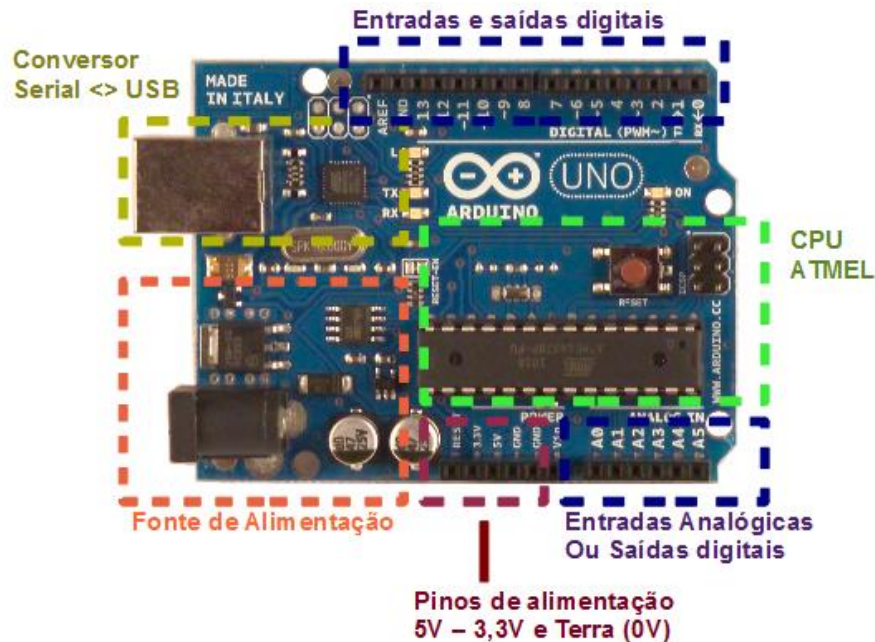
Fonte: <https://store.arduino.cc/usa/physics-lab>

Fonte: <https://www.fosteropenscience.eu/content/what-open-science-introduction>



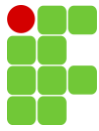
2. Estrutura do Arduino

A placa de **UNO R3**, na qual iremos programar possui um conjunto de portas especificadas para cada componente ou sensor que será conectado. Abaixo temos um resumo das principais portas.



Fonte: www.arduino.org

- **Portas digitais** recebem sinais digitais, isto é, 1 (um) ou 0 (zero). Constituída por 14 portas.
- **Portas analógicas** recebem sinais escalares, por exemplo: distância de um objeto, a luminosidade de um ambiente, o índice de acidez e de uma solução, etc. Constituída por 6 portas.
- **Porta de alimentação:** onde podemos conectar nossa placa a uma fonte de alimentação comum de até 12 V.
- **Porta USB:** porta de comunicação e transmissão do código para o microcontrolador.



3. Sensores, Atuadores e Módulos

Para o desenvolvimento de experimentos é importante compreender a diferença entre **sensores**, **atuadores** e **módulos**.

Sensores são dispositivos que captam ou coletam informações e transmitem para a placa em forma de sinais elétricos, convertidos posteriormente em bits, por exemplo:

		
Sensor de temperatura	Sensor de luminosidade	Sensor Magnético reed switch

Atuadores são dispositivos que emitem informações. São componentes eletrônicos que podem ser controlados ou manipulados pelo desenvolvedor. Por exemplo:

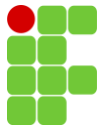
		
Buzina	Led	Motor

Módulos são constituídos por **Circuitos Integrados (CI)**, pequenas placas conectadas tanto por **sensores** como por **atuadores**. Os módulos possuem algumas funcionalidades para ajustes ou equalização dos dados, por exemplo.

			
Módulo ultrassônico	Módulo De cor	Módulo de som	Módulo display 7 segmentos

Existem ainda os módulos de transmissão, por exemplo: **Módulo Bluetooth**.



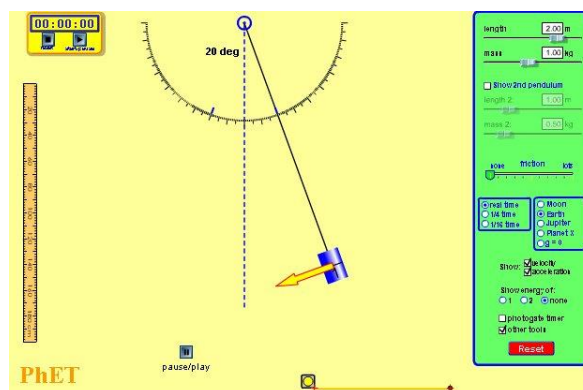


4. Diferença entre Emulador e Simulador

Simulação:

Um ‘simulation’ é um sistema (software, programa, aplicativo) que se comporta semelhante a outro aparato (instrumento, equipamento, ferramenta, etc.), mas é implementado de uma maneira totalmente diferente.

Fornece o comportamento básico de um sistema, mas pode não necessariamente aderir a todas as regras do sistema que está sendo simulado. O simulador dar uma ideia de como algo funciona, um exemplo são os simuladores do site PHET.



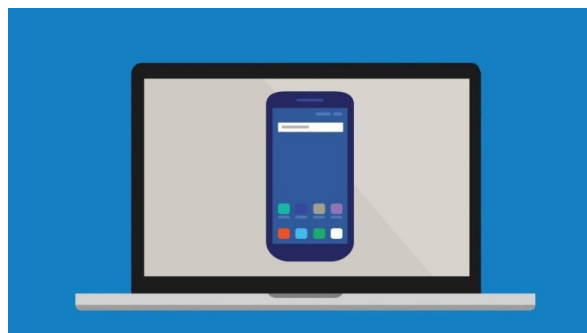
Simulador de Pêndulo

Fonte: <http://fisicanimada.blogspot.com/2008/12/fsica-do-pndulo.html>

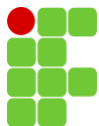
Emulador:

Um ‘emulation’ é um sistema (software, programa, aplicativo) que se comporta exatamente como outro aparato (instrumento, equipamento, ferramenta, etc.), e segue todas as regras do sistema que está sendo emulado.

Efetivamente é uma **replicação** completa de outro sistema, sendo compatível com as entradas e saídas do sistema emulado, mas operando em um ambiente diferente do ambiente do sistema original emulado. As regras são fixas e não podem ser alteradas ou o sistema falha.



Emulador de dispositivo móvel

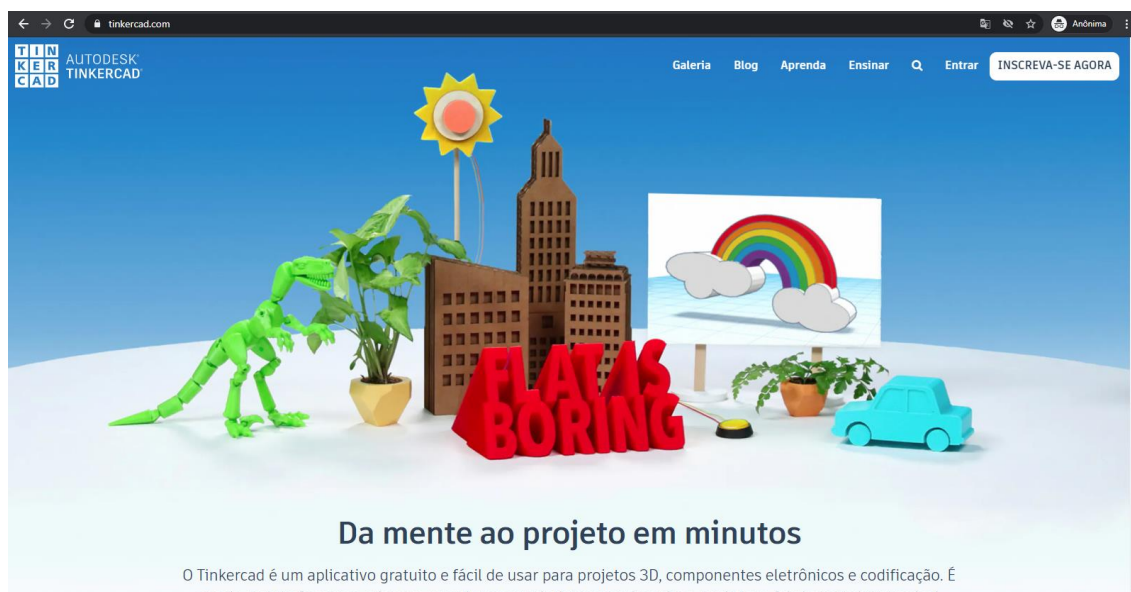


5. A plataforma TINKERCAD

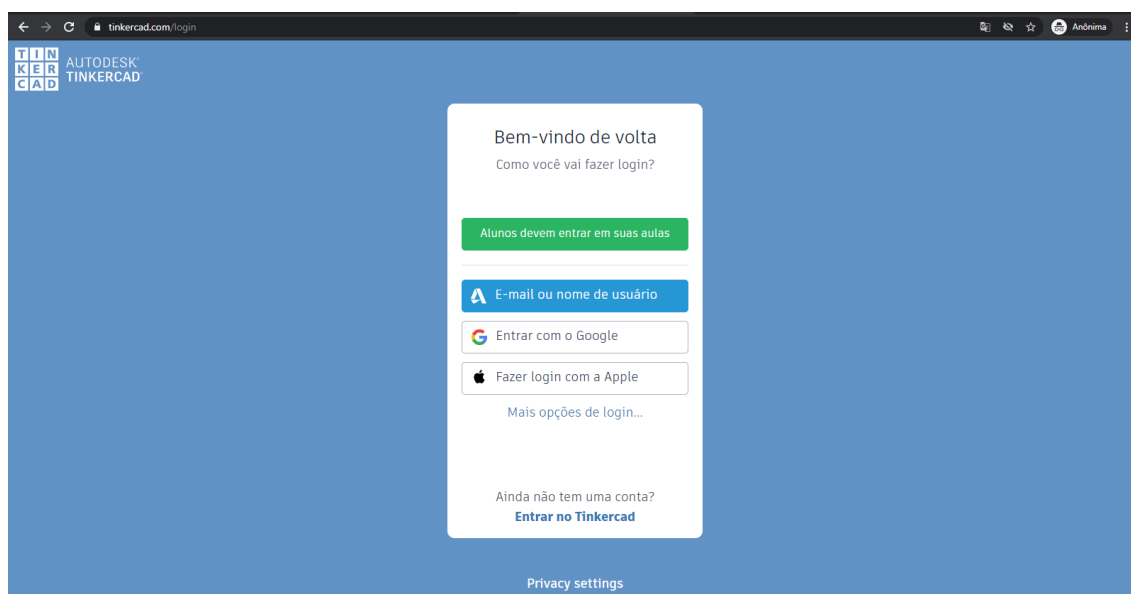
Constitui-se de um emulador virtual, com praticamente todas as características e funcionalidades da placa Arduino, assim como do seu ambiente de programa. No **Tinkercad** os projetos modelados podem ser **Privados** ou **Públicos**, cabendo somente ao desenvolvedor disponibilizar suas criações.

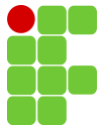
O **Tinkercad** possui integração com o **Google Classroom**, ambiente virtual de aprendizagem disponibilizado pela Google.

Para usufruir da plataforma é necessário acessar o endereço: <https://www.tinkercad.com>, e realizar um cadastro prévio.

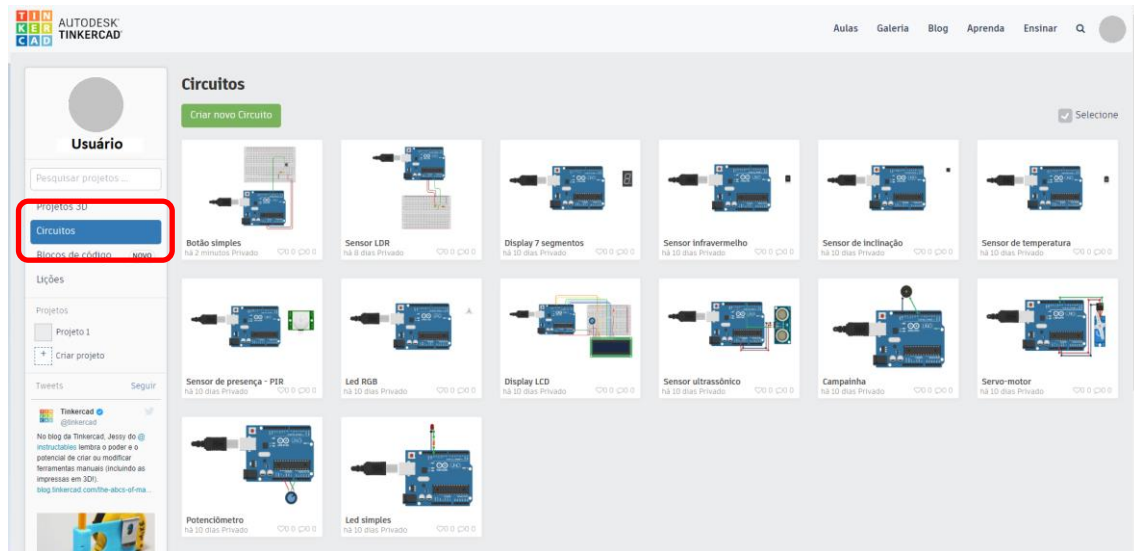


Seleciona-se uma conta e o cadastro é efetivado.





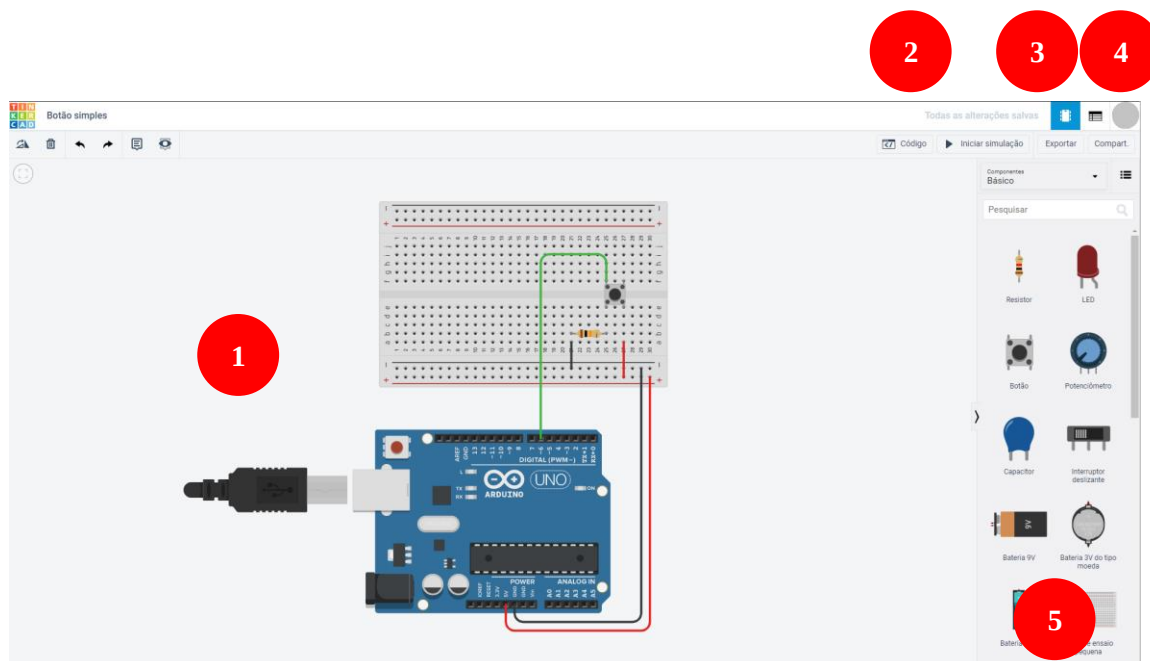
Com o cadastro do usuário efetuado a **Tela inicial** abaixo é apresentada.

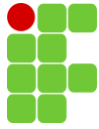


Para criar um circuito, selecione a opção: **Circuitos** em seguida clique no botão: **Criar novo Circuito**.

O usuário pode ainda criar um repositório dos circuitos selecionando a opção: **Criar projeto**. Separando os projetos conforme tipos e modelos.

Quando um circuito é criado será apresentada a tela do emulador abaixo com os itens de **modelagem (1)**, **programação (2)**, **exportação de código (3)**, **compartilhamento (4)** e **componentes eletrônicos (5)**.





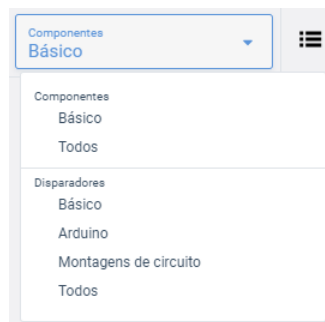
Na **área central** temos o ambiente de **montagem** e **conectorização** dos componentes eletrônicos. Ao inserir um componente podemos:

- Alteração sua direção (1).
- Apagar (2).
- ‘Fazer’ uma ação (3).
- ‘Desfazer’ uma ação (4).
- Inserir um comentário (5).
- Exibir/ocultar um componente (6).



Para ajustar um componente no centro da tela de modelagem selecione a opção:

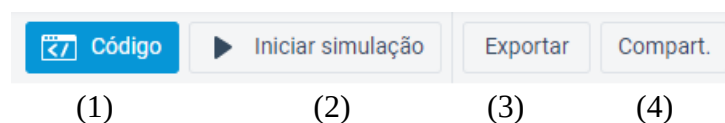
Dentro da tela de emulador, do **lado direito**, temos à aba **listagem de componentes**. Dividida em **Componentes** e **Disparadores**.

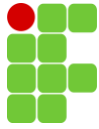


Na opção **Arduino** (na aba: Componentes) encontram-se diversos modelos de projetos já modelados. Nestes podemos compreender, de forma simplificada, o funcionamento das conexões, sensores e demais componentes.

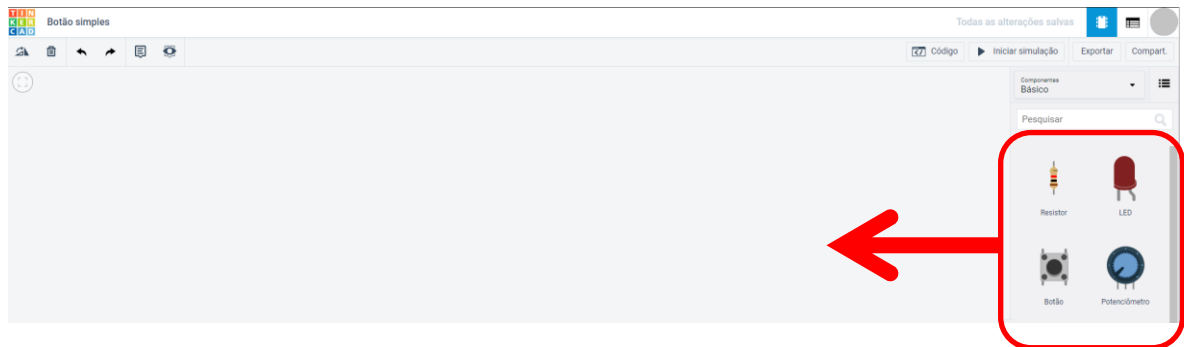
Acima da **listagem de componentes** temos a **barra de programação**. Constituída por quatro botões:

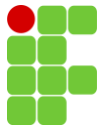
- Ambiente programação (1).
- Acionamento do emulador (2).
- Exportação do código (3).
- Compartilhamento do projeto (4).





Para modelar um projeto, basta selecionar um componente e clicar na **área de modelagem**.

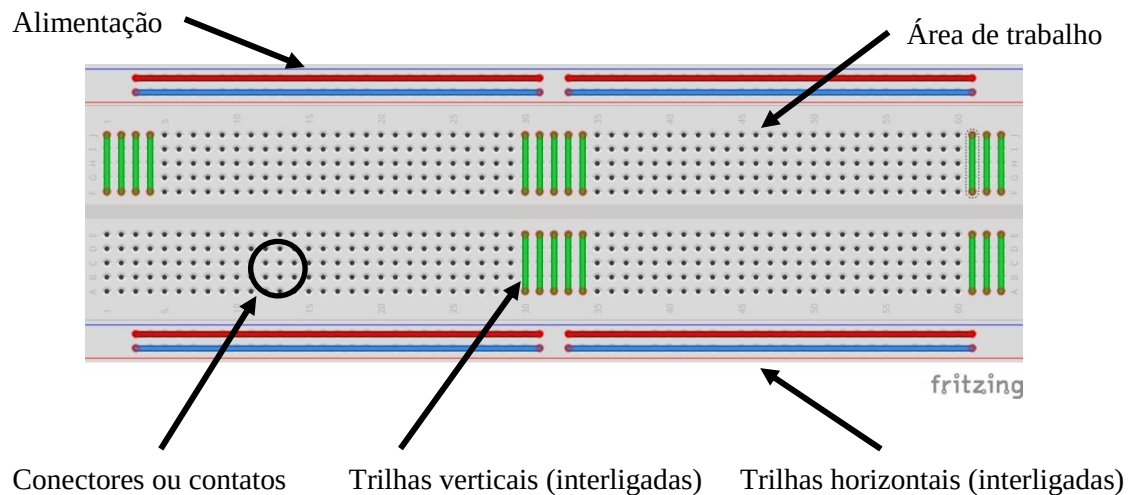




6. A Protoboard

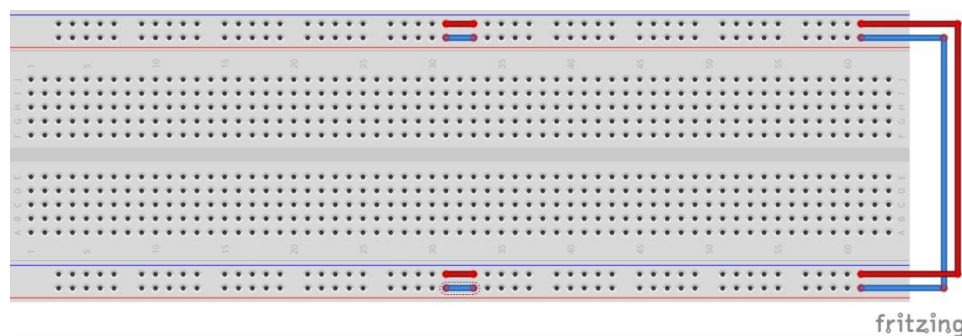
Componente fundamental para as montagens eletrônicas e prototipagem dos projetos, a **protoboard** (também denominada **breadboard**) ou **matriz de contatos** é de fácil utilização, contudo é necessário compreender a organização de suas trilhas.

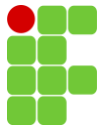
Na imagem abaixo indicamos os elementos pertencentes a matriz de contatos.



- Uma **protoboard** possui 4 conjuntos de trilhas horizontais para alimentação. Geralmente indicada de azul ou preto, esta trilha é denominada de **Neutro, Terra, GND** ou **Ground**. A outra indicada de vermelho é utilizada para energia ou voltagem.
- Os furos são utilizados para conectar componentes e cabos (jumpers).
- A área de trabalho é o espaço onde realizamos a montagem do projeto.
- As linhas verdes indicam a forma como são conectadas as trilhas verticais.

Em uma **protoboard** as trilhas de alimentação não estão interligadas. O projetista, caso utilize uma protoboard maior deve conecta as trilhas de alimentação horizontais. Abaixo apresentamos uma sugestão de conectividade.

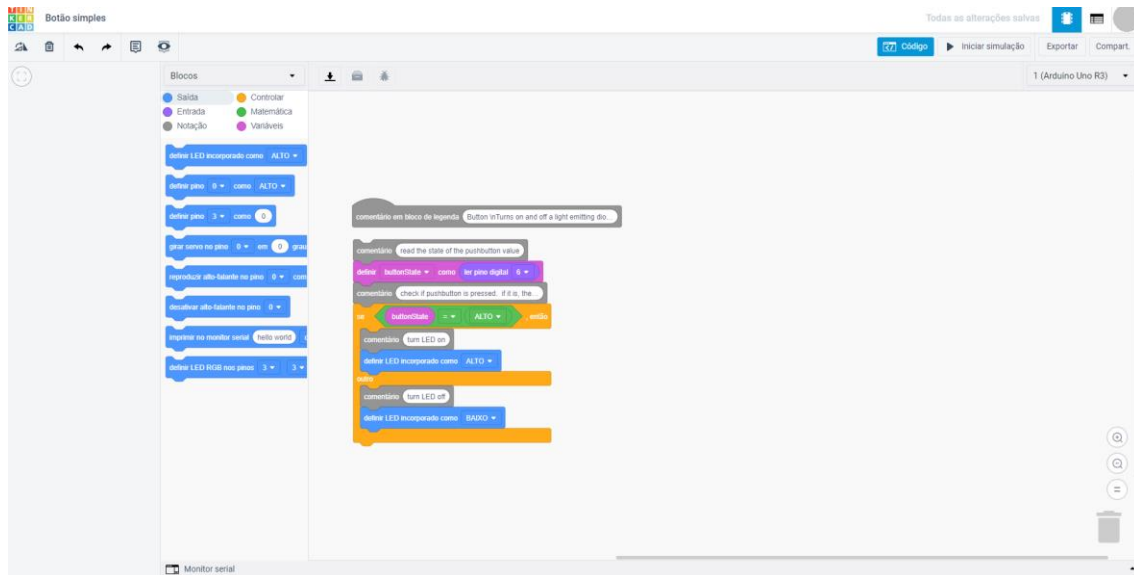




7. A programação

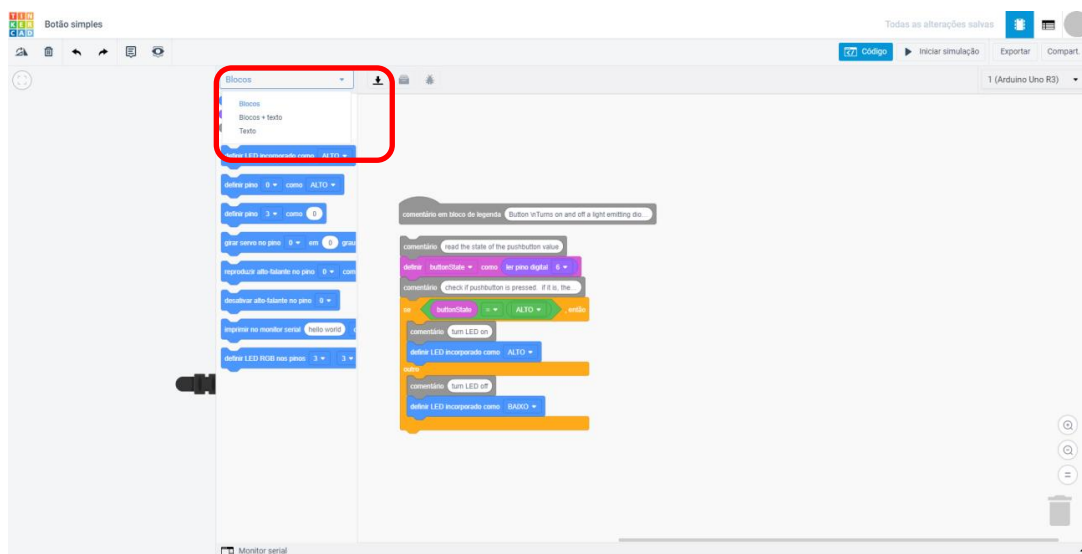
Para programar selecione a opção  **Código**. Uma tela será exibida apresentando os **seis conjuntos** de comandos que o emulador possui em formato de **Blocos**.

O número de instruções existentes na opção de **Blocos** é limitado a alguns sensores e dispositivos.

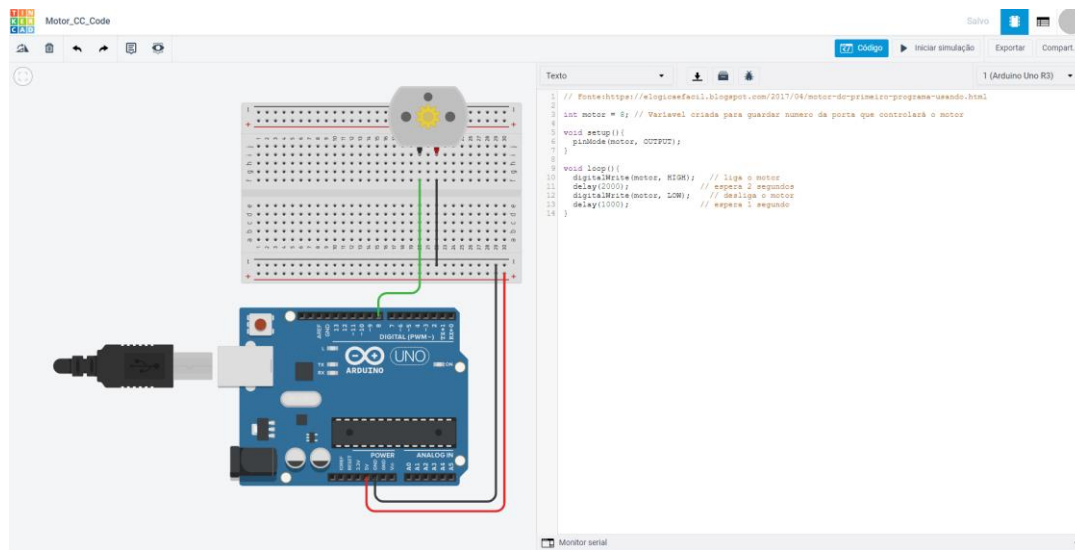
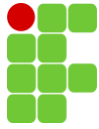


Ainda no ambiente de programação podemos optar por escolher entre código em **Blocos**, **Blocos com Texto** ou somente código em **Texto**.

Na opção **TEXTO** existem várias outras instruções que permitem um melhor controle dos componentes e dos dispositivos. Assim como inserir comandos nativos da linguagem de programação da placa Arduino.



Tela somente Blocos

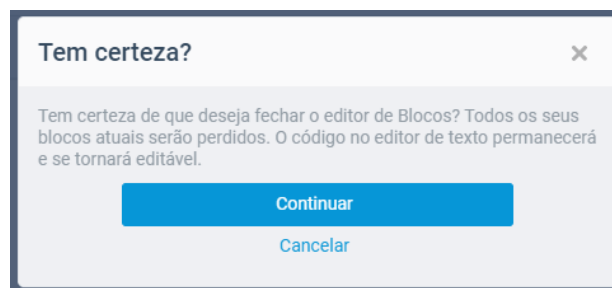


Tela somente Texto

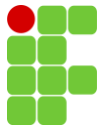
Podemos encontrar uma listagem com todas as instruções no endereço: <https://www.arduino.cc/reference/pt/#functions>.

Observação:

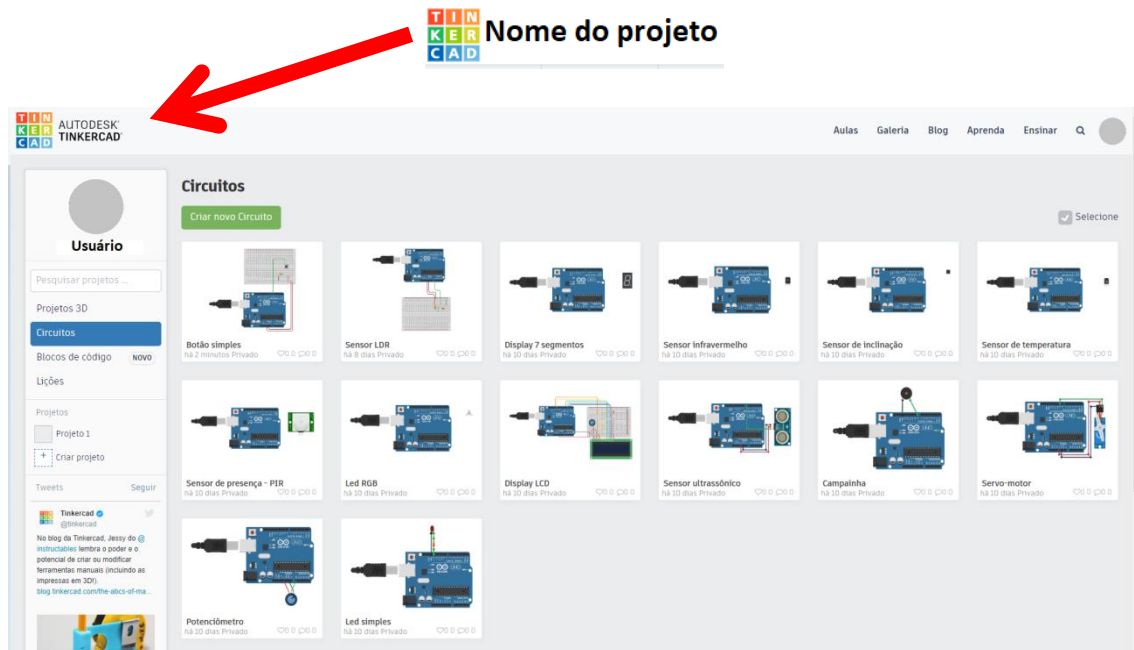
Caso seja alterado de um **código em bloco** para um **código em texto**, a seguinte mensagem será apresentada. Seu código será pedido e não haverá forma de recuperá-lo. O mesmo irá ocorrer caso seja trocado do **código em texto** para o **código em bloco**.



Todos os códigos implementados em um projeto serão armazenados junto à modelagem eletrônica.

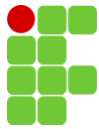


Quando um projeto for salvo, será criada uma miniatura na **tela inicial**. Para retornar a **tela inicial** sempre que necessário basta clicar na logo do **TINKERCAD**, no canto superior esquerdo.



Observação:

O ambiente salva automaticamente todo projeto iniciado com periodicidade contínua. O projetista não controla o tempo de salvamento.

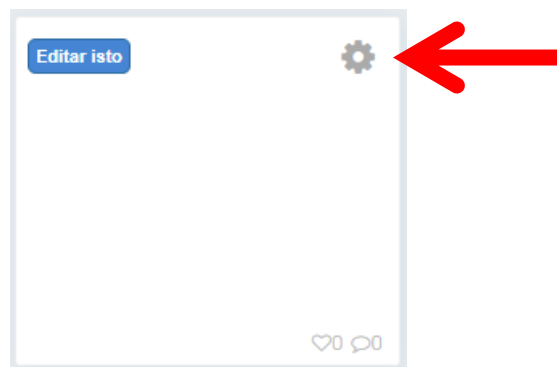


8. Duplicando, Editar e Mover um circuito

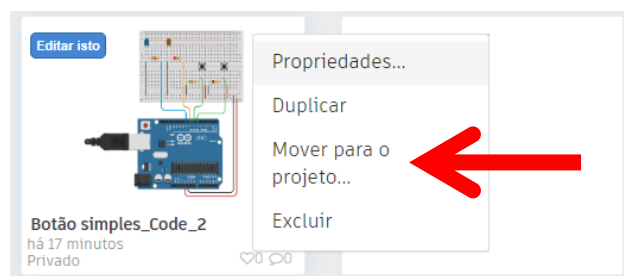
Para Editar um projeto basta clicar no ícone em azul .

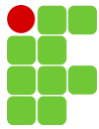


Para duplicar um circuito clicar no ícone da **engrenagem**  e selecionar a opção **Duplicar**.



Para mover um circuito clicar no ícone da **engrenagem**  e selecionar a opção **Mover para o Projeto**.



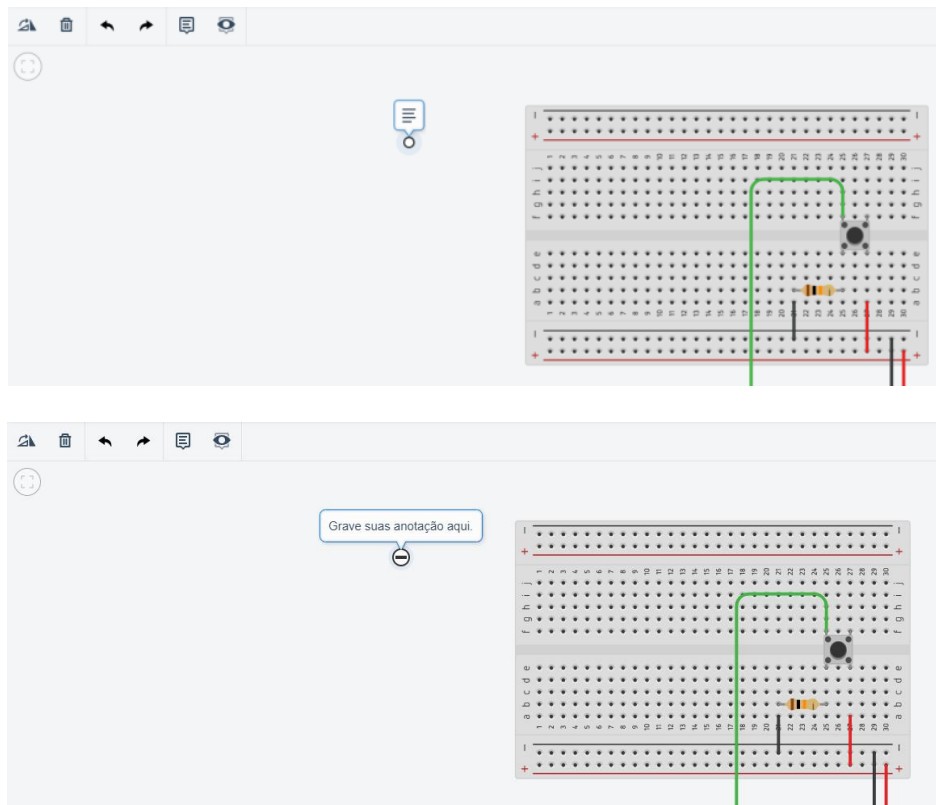


9. Rótulos para comentários

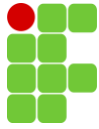
Documentar cada passo ou etapa de um algoritmo no emulador é de fundamental importância. Inserir comentários que informem o que cada instrução irá executar permite compreender seu funcionamento e sua lógica. No código podemos comentar em formato de bloco ou somente uma frase.

Frase	// Blink: acende um LED por um segundo, depois apaga por um segundo.
Bloco	/* Blink Acende um LED por um segundo, depois apaga por um segundo */

Detalhar um **programa** ou um **projeto** com comentários é considerado uma ótima prática, assim como permite que o código ou a modelagem sejam revisados facilmente. Na área de modelagem utilizamos o ícone: , da **barra de ferramentas**.

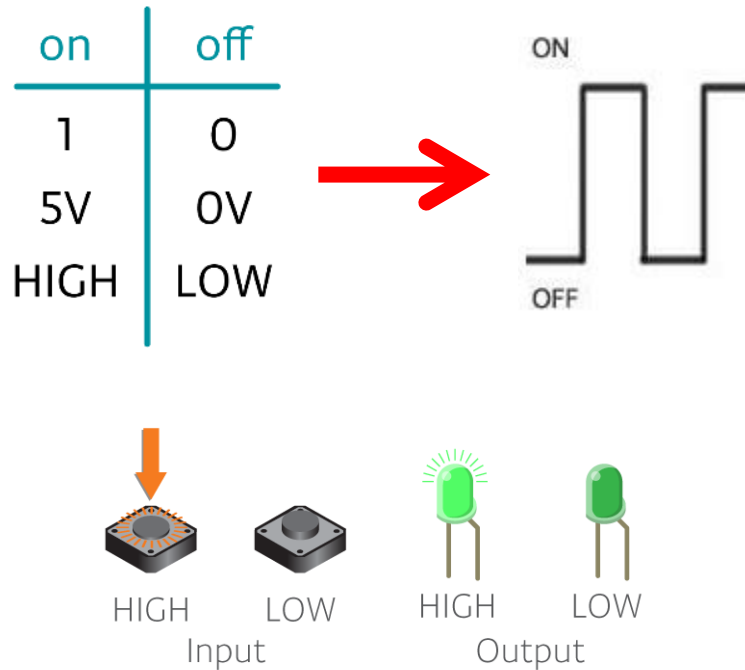


Comentário na modelagem



10. Os indicadores: LOW e HIGH

Quando estabelecemos um valor **ALTO** significa uma situação de **HIGH**; quando escolhemos **BAIXO**, significa **LOW**, tanto para informações que estão sendo inseridas/captadas (Input), quando aquelas que estão sendo transmitidas (Output)

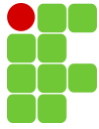


Um botão acionado significa que uma corrente ou uma informação será transmitida, seu estado será considerado **HIGH**. O mesmo pode ocorrer quando um LED for acionado através de uma instrução.

Fonte:

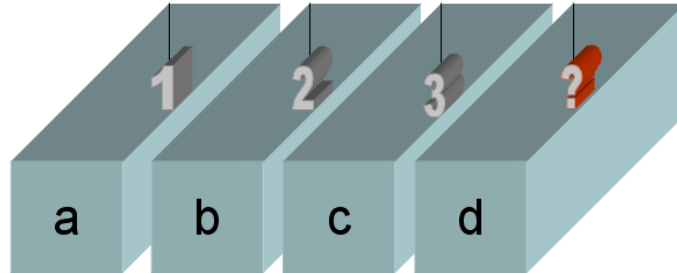
<https://www.luisllamas.es/entradas-analogicas-en-arduino/>

<https://www.luisllamas.es/entradas-digitales-en-arduino/>



11. Criação de variáveis

As denominadas **variáveis** são ‘espaços’ ou ‘caixas’ que servem para guardar temporariamente valores ou dados. Seu uso é constante em praticamente todos os projetos.



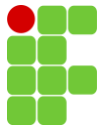
Por exemplo: se desejamos guardar os valores da **temperatura** de dois sensores, e depois calcular sua média, serão necessárias três variáveis: **ValorTemp1**, **ValorTemp2** e **ValorTempMedia**.

Caso, ainda no código, desejarmos identificar a maior e a menor temperatura. Teremos que criar as seguintes variáveis: **ValorTempMax** e **ValorTempMin**, respectivamente.

Observação:

O nome de uma variável fica a critério do seu projetista, contudo é considerada uma boa prática nomes que possam identificar facilmente a finalidade daquela nova instrução, por exemplo:

- Sensor ultrassônico: **ValorUltrasom**.
- Potenciômetro: **ValorPonten**.
- Ângulo de inclinação: **ValorInclinar**.



12. Monitores de dados

O emulador possui dois tipos de monitores de dados: **Serial Monitor** e o **Serial Plotter**.



Serial Monitor

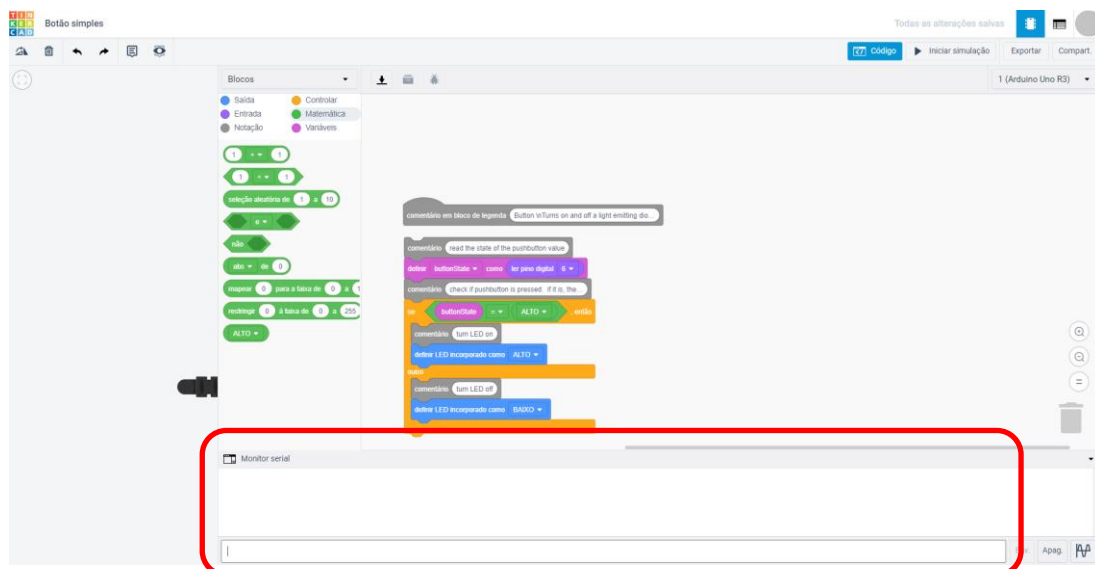


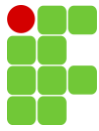
Serial Plotter

O **Serial Monitor** irá apresentar valores numéricos de **Entrada**, **Saída** ou das **Variáveis** criadas no projeto.

A visualização do resultado de cálculos também pode ser exibidos no **Serial Monitor**, bem como **frases** e/ou **gatilhos** indicadores de qualquer ação implementada no código.

Por exemplo: utilizando um **sensor de luminosidade**, quando a valor ultrapassar determinada regra, isto é, uma condição pré-definida, aparecerá no Monitor Serial uma frase informando que foi atingido certo limite.

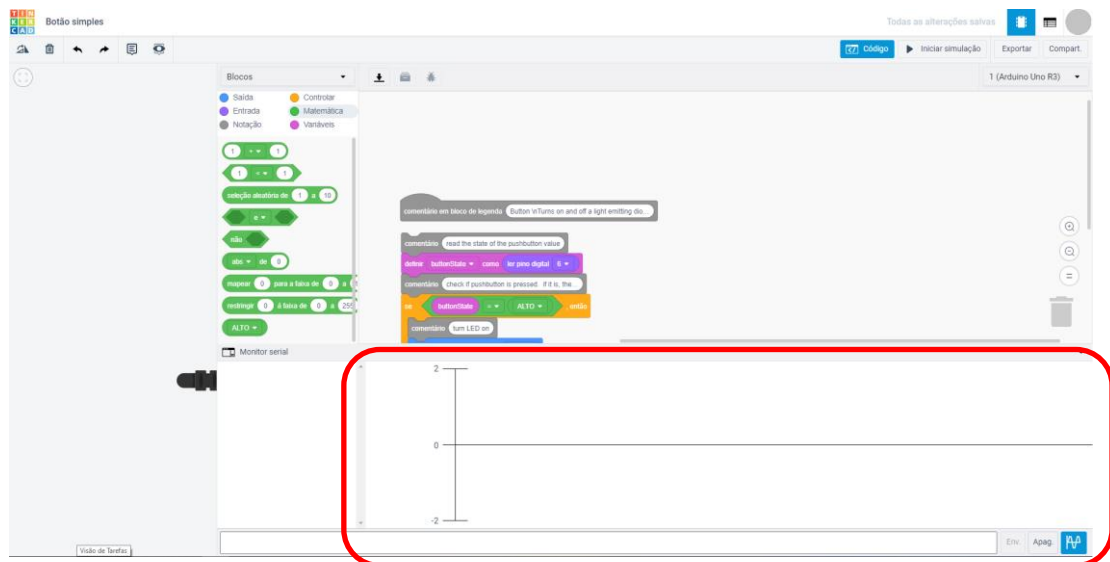




Outro monitor disponível no emulador é o **Monitor Plotter**.

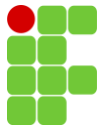
Acionando o botão Gráfico , no canto direito inferior, é exibido **dois eixos de coordenadas**, semelhante a um plano cartesiano.

Os valores coletados em um experimento serão exibidos em forma **gráfica**.



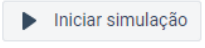
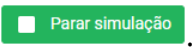
Observação:

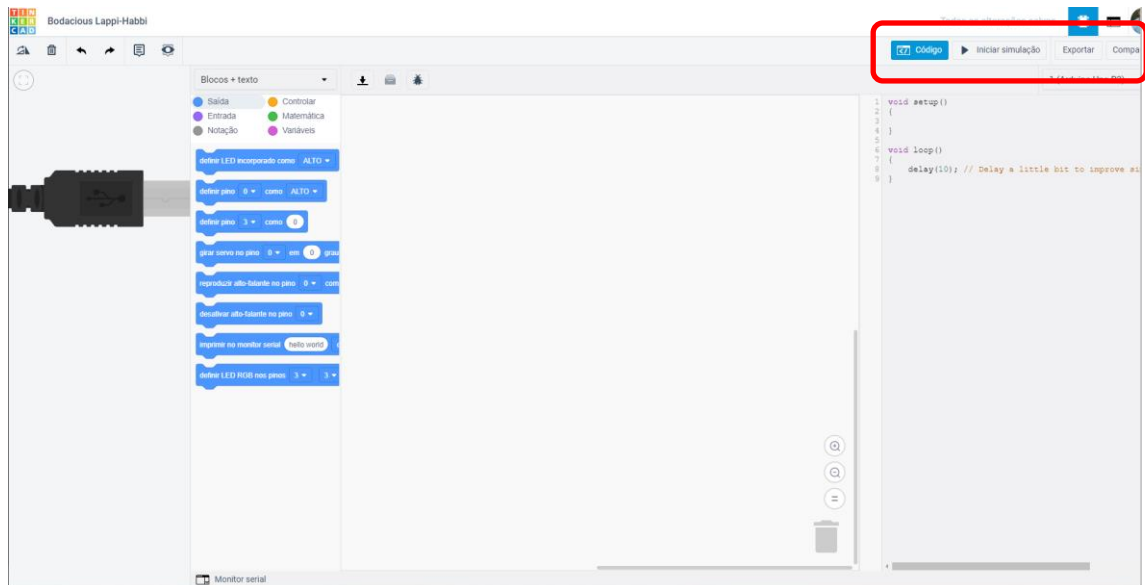
O emulador Arduino não dispõe de recursos ou funcionalidades que armazenem os dados coletados tanto no **Serial Monitor**, quanto no **Serial Plotter**.

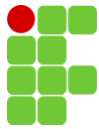


13. Acionando um emulador

Ao concluir um projeto podemos testar a modelagem e o código, **ligando** ou **desligando** o botão de emulação na **barra de programação**.

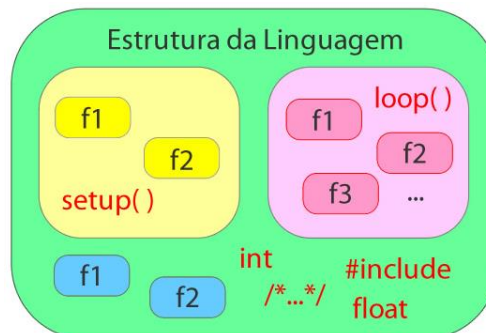
- Emulador desligado:  .
- Emulador ligado:  .





14. Estrutura de um programa ou sketch Arduino

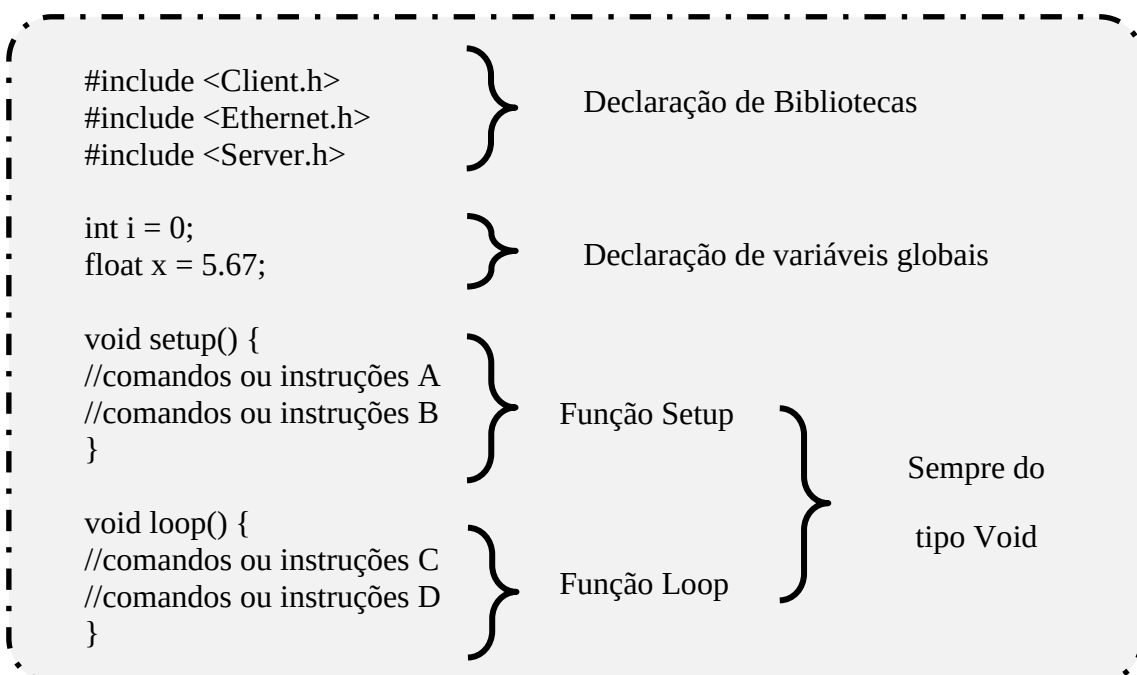
Todo programa desenvolvido possui uma organização bem simples, dividida geralmente em **cinco “regiões”**. Cada um desses segmentos possui uma tarefa específica. São elas conforme imagem abaixo:

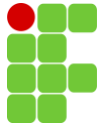


Fonte: <http://www.ordemnatural.com.br>

- Região para inclusão das **bibliotecas** (no início do programa, utiliza o caractere ‘#’ como indicador).
- Espaço para declaração das **variáveis** e **constantes** globais.
- Função **Setup** (responsável por ‘carregar’ o endereçamento das portas utilizadas pelo Arduino no projeto)
- Função **Loop**, área no qual escrevemos o programa que será executado infinitamente.

Há ainda uma quinta área, denominada de **FUNÇÃO**, na qual o programador pode criar novos códigos quando for necessário. Sua anatomia e operacionalidade é semelhante as funções **Setup** e **Loop**.





15. Referências

- Fundação Arduino e Genuino: www.arduino.cc
- Oficial Arduino: www.arduino.org
- Loja Filipeflop: www.filipeflop.com.br
- Blog loja Filipeflop: blog.filipeflop.com
- Como Fazer as Coisas: www.comofazerascosas.com.br
- Fritzing: <http://fritzing.org>
- Laboratório de Garagem: <http://labdegaragem.com>
- rduino & Cia: www.arduinoecia.com.br/
- Loja GBK Robotics: <http://gbkrobotics.com.br>